00 01 02 03 04 05 **06** 07 08 09 0A 0B 0C 0D

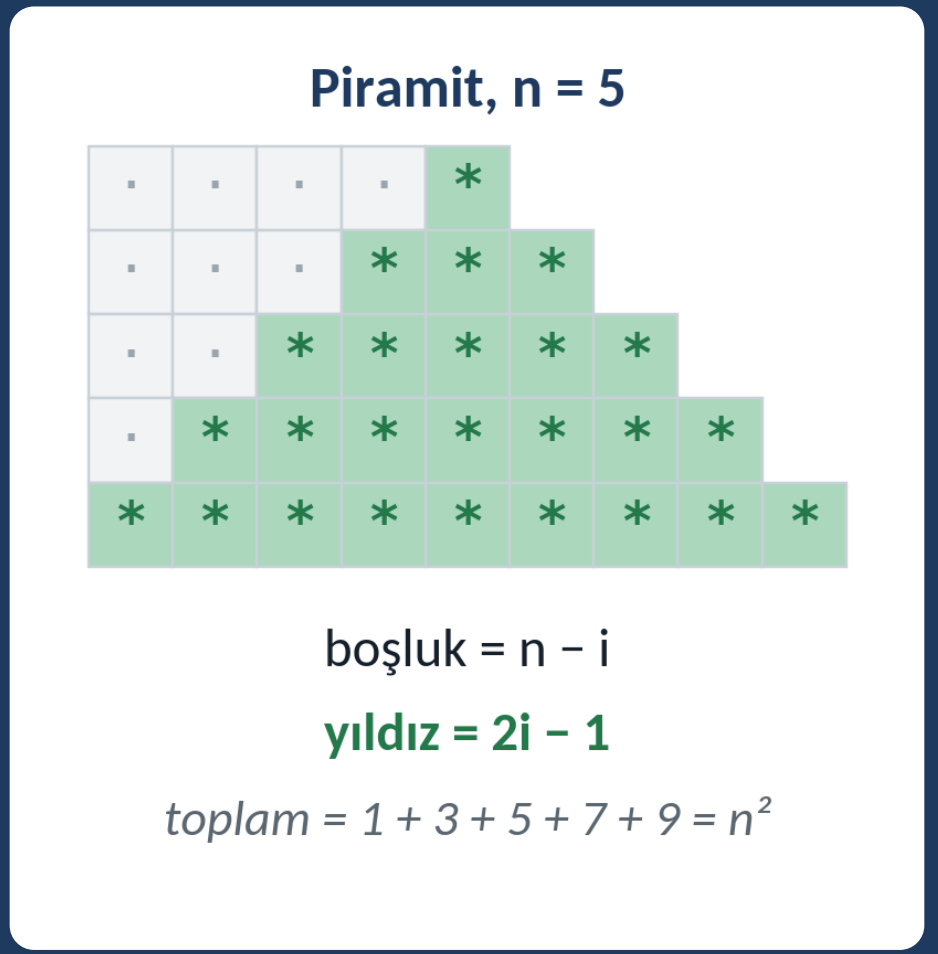
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 07

Döngüler II: İç İçe Döngüler, Problem Çözme Desenleri ve Ara Proje

Izgara modeli ve yineleme sayısı · satır–sütun desenleri · asallık ve √n · seriler ve ε · clock() ile ölçüm · menü döngüsü ve ATM

MODÜL III · KONTROL AKIŞI



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** **Döngüler II + Ara Proje**
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H6

Geçen haftadan · S1: Bu program ne yazdırır?

quiz/H07_Q01_dongu_turleri.c · satır 10–23

```c
10      int a = 5;
11      int b = 5;
12      while (a < 3) {
13          printf("W");
14          a++;
15      }
16      do {
17          printf("D");
18          b++;
19      } while (b < 3);
20      for (int c = 0; c < 3; c += 2) {
21          printf("F");
22      }
23      printf(" %d %d\n", a, b);
```

A 5 5 (hiçbir gövde çalışmaz)

B DFF 5 6

C WDFF 6 6

D DFFF 5 6

TAHMİN ET

Geçen haftadan · S2 ve S3

S2 (nöbetçi değer): girdi `4 7 -1 9` ise çıktı nedir?

`9` neden toplanmaz?

quiz/H07_Q02_nobetci_toplam.c · satır 10–18

```c
    int x;
    int toplam = 0;
    int adet = 0;
    printf("Sayılar (-1 ile bitir): ");
    while (scanf("%d", &x) == 1 && x != -1) {
        toplam += x;  /* toplayıcı */
        adet++;       /* sayaç */
    }
    printf("adet = %d, toplam = %d\n", adet, toplam);
```

S3 (scanf dönüşü): `abc` yazılırsa `r` ve `n`?

Ctrl+D (dosya sonu) basılırsa?

quiz/H07_Q03_scanf_donus.c · satır 10–13

```c
    int n = 7;
    printf("Bir sayı: ");
    int r = scanf("%d", &n);
    printf("r = %d, n = %d\n", r, n);
```

GEÇEN HAFTADAN

Yanıtlar: gerçek çıktılar

S1 · B

GERÇEK ÇIKTI

```
$ ./q01
DFF 5 6
```

- `while`: 0 tur · `do-while`: 1 tur
- `for`: c = 0, 2 → 2 tur

S2 · nöbetçi

GERÇEK ÇIKTI

```
$ ./q02
Sayılar (-1 ile bitir): 4 7 -1 9
adet = 2, toplam = 11
```

- -1 okununca döngü biter
- `9` hiç okunmaz

S3 · dönüş değeri

GERÇEK ÇIKTI

```
$ ./q03
Bir sayı: abc
r = 0, n = 7

$ ./q03
Bir sayı: r = -1, n = 7
```

- `abc` → 0, `n` değişmez
- EOF → -1 (`EOF`)

Bu hafta: döngünün içinde döngü → **ızgara**, **desen**, **arama** ve **menü**

GERÇEK DÜNYADAN

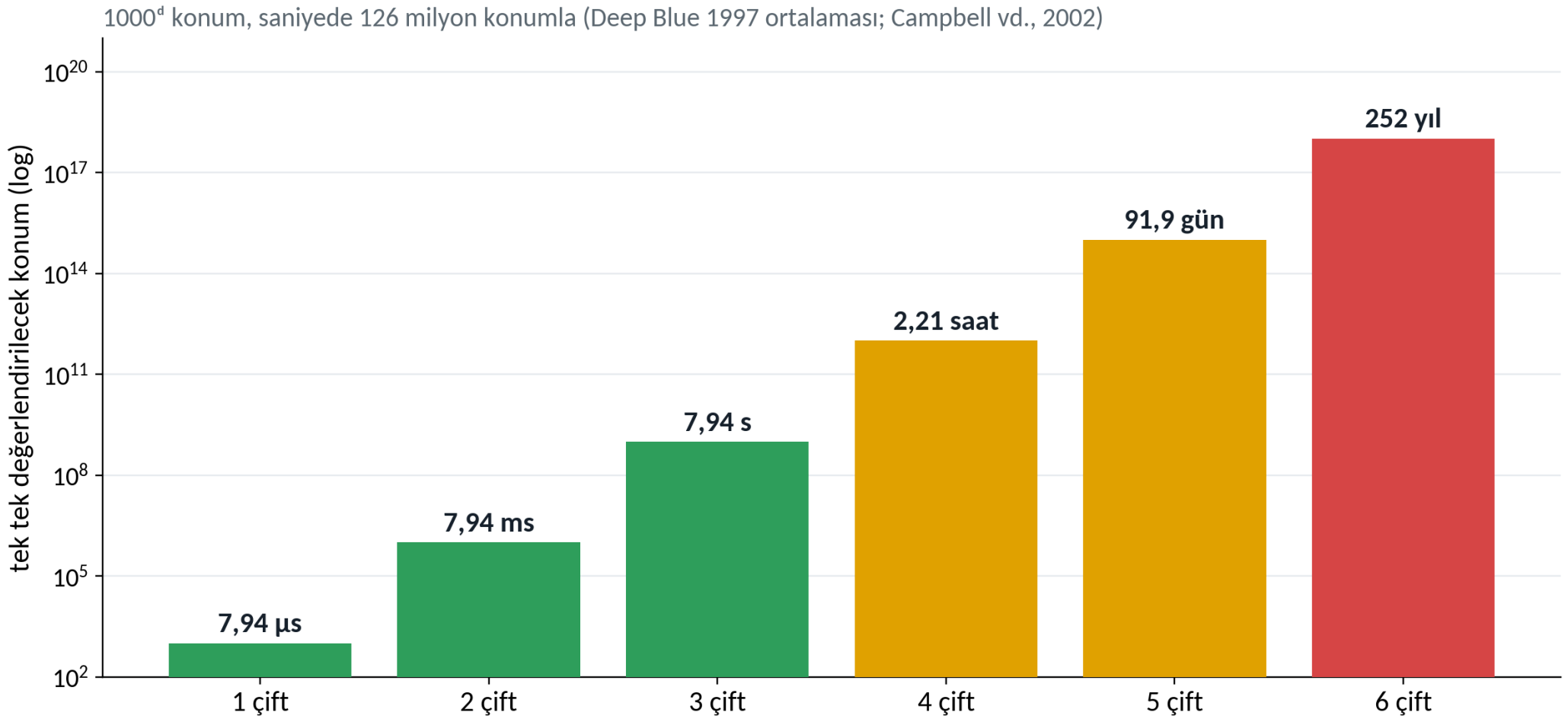
Mayıs 1997

Deep Blue, Kasparov'u yendi: kaba kuvvetin gücü

- Mayıs 1997, New York: IBM'in Deep Blue'su altı oyunluk rövanşı **3½–2½** kazandı (IBM, t.y.)
- 30 düğümlü RS/6000 SP + **480 satranç yongası**; ortalama **saniyede 126 milyon konum** (Campbell vd., 2002)
- Saf "her şeyi dene" değildi: donanımda değerlendirme fonksiyonu ve **seçici arama uzatmaları**
- Çünkü kaba kuvvetin sınırı var: olasılıklar **çarpılarak** büyür

Campbell, Hoane & Hsu (2002), Artificial Intelligence 134; IBM (t.y.), Deep Blue

Kaba kuvvetin sınırı: arama uzayı çarpılarak büyür



Shannon'ın (1950) kaba kestirimi: bir hamle çiftinde ~1000 olasılık · süreler 126 milyon konum/s ile (gerçek çıktı)

BÜYÜK SORU

Haftanın Büyük Sorusu

Bir sayının asal olup olmadığını en hızlı nasıl anlarız ve “daha hızlı” iddiamızı nasıl ölçeriz?

455 189 150

bölme: naif test, 0..100 000

1 345 442

bölme: √n + tek bölenler

- İki ölçü: makineden bağımsız **yineleme sayısı** (sayaç) ve makineye bağlı **süre** (`clock()`)
- Yanıtı hafta sonunda, kendi ölçümümüzle vereceğiz

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 7.1	İç içe döngülerin çalışma sırasını izler ve toplam yineleme sayısını hesaplar.
ÖÇ 7.2	Satır–sütun düşüncesiyle iki boyutlu çıktı desenleri üretir.
ÖÇ 7.3	Sayı kuramı problemlerini (asallık, mükemmel sayı, Armstrong sayısı) döngülerle çözer; çözümü verimlilik açısından iyileştirir (√n sınırı).
ÖÇ 7.4	clock() ile çalışma süresini ölçerek iki çözümü deneysel olarak karşılaştırır.
ÖÇ 7.5	Menü güdümlü, kullanıcıyla etkileşimli bir program tasarlar ve gerçekler.

01

İç içe döngü ve satır–sütun düşüncesi

ızgara modeli · yineleme sayısı n·m ve n(n+1)/2 · desenleri formülle çizmek

İç içe döngü: dış döngünün her turunda iç döngü baştan sona

H07_K01_izgara_sayac.c · satır 17–23

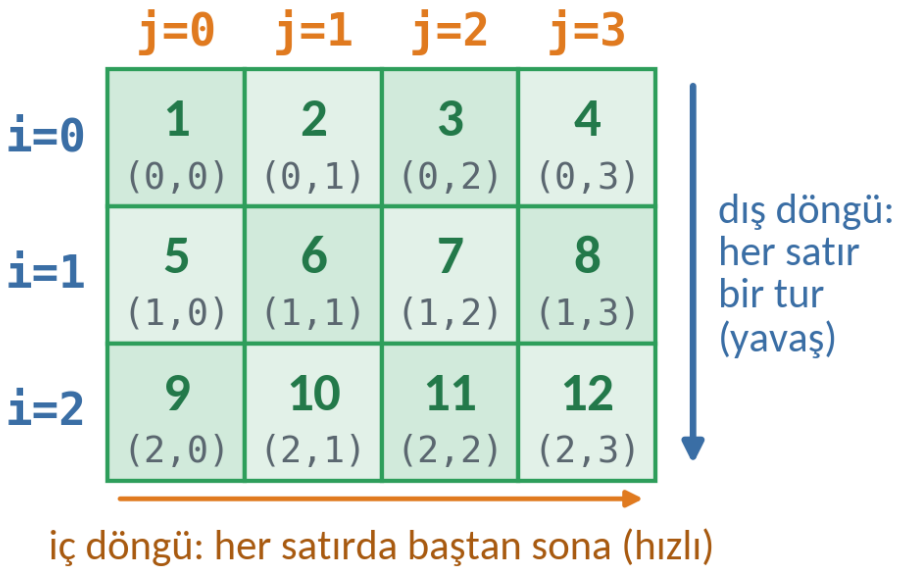
```
17      for (int i = 0; i < SATIR; i++) {      /* dış: yavaş değişir */
18          for (int j = 0; j < SUTUN; j++) {  /* iç: hızlı değişir */
19              adim++;
20              printf("(%d,%d) ", i, j);
21          }
22          printf("<- i = %d satırı bitti\n", i);
23      }
```

ÇIKTI

```
$ ./izgara_sayac
(0,0) (0,1) (0,2) (0,3) <- i = 0 satırı bitti
(1,0) (1,1) (1,2) (1,3) <- i = 1 satırı bitti
(2,0) (2,1) (2,2) (2,3) <- i = 2 satırı bitti
İç gövde 12 kez çalıştı = 3 x 4
```

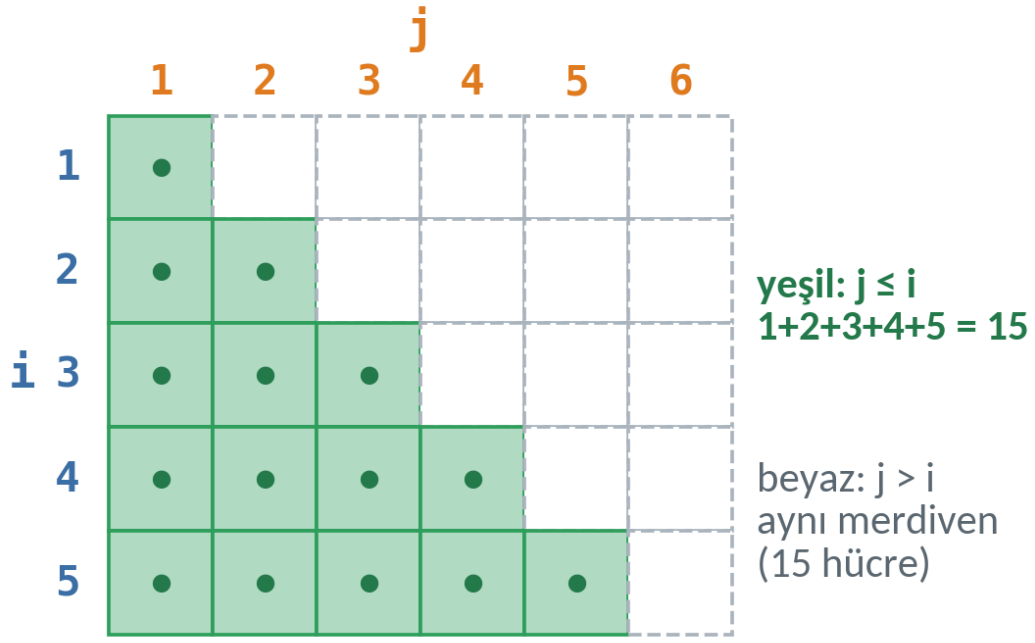
Izgara modeli: her hücre bir yineleme

```
for (i = 0; i < 3; i++)
    for (j = 0; j < 4; j++)
```



toplam yineleme = 3 × 4 = 12 (n · m)

```
for (i = 1; i <= 5; i++)
    for (j = 1; j <= i; j++)
```



iki merdiven = 5 × 6 dikdörtgen ⇒ 15 = n(n+1)/2

Ders notu Şekil 7.1: solda 3 × 4 dikdörtgen, sağda j ≤ i üçgeni ve onu 5 × 6 dikdörtgene tamamlayan ikinci merdiven

TAHMİN ET

Kaç kez çalışır? sayac ne olur?

quiz/H07_Q11_ic_ice_sayac.c · satır 10–16

```
10      int sayac = 0;
11      for (int i = 0; i < 4; i++) {
12          for (int j = 0; j < 6; j++) {
13              sayac++;
14          }
15      }
16      printf("%d\n", sayac);
```

A 10

B 24

C 35

D 20

Kaç yineleme? Sayaçla ölçtük, formülle karşılaştırdık

N on katına çıkınca: iki katlı döngü **yüz kat**, üç katlı döngü **yaklaşık bin kat** daha çok yineleme

N	A) j ≤ N	B) j ≤ i	C) j < i	D) a ≤ b ≤ c ≤ N
10	100	55	45	220
100	10 000	5 050	4 950	171 700
1000	**1 000 000**	**500 500**	**499 500**	**167 167 000**
formül	N^2	$N(N+1)/2$	$N(N-1)/2$	$N(N+1)(N+2)/6$

Desenleri formülle çizmek: dört adım

1

Kâğıda çiz

Çıktıyı n = 4 için kareli kâğıda çiz; boşlukları · ile işaretle

→

2

Satır tablosu

Her satır için: `i`, baştaki boşluk, yıldız sayısı

→

3

Formül

Sütunları `i` ve `n` ile yaz; ilk ve son satırda **sına**

→

4

Kod

Dış döngü satırlar; içinde formül başına **bir** iç döngü; sonda `\n`

Uçbirim geri dönemez: her satır soldan sağa, bir kez yazılır

ÖRNEK 4 · AMİRAL

Piramit: boşluk = n – i, yıldız = 2i – 1

H07_04_piramit.c · satır 21–30

```
21      for (int i = 1; i <= n; i++) {
22          for (int b = 1; b <= n - i; b++) {  /* önce n - i boşluk */
23              printf(" ");
24          }
25          for (int y = 1; y <= 2 * i - 1; y++) {  /* sonra 2i - 1 yıldız */
26              printf("*");
27              toplam++;
28          }
29          printf("\n");
30      }
```

GERÇEK ÇIKTI · N = 5

```
$ ./piramit
> 5
    *
   ***
  *****
 *******
*********
Toplam 25 yıldız (n² = 25)
```

i	boşluk n – i	yıldız 2i – 1
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

- Dış döngünün gövdesinde **art arda iki** iç döngü: önce boşluk, sonra yıldız (iç içe değil, sıralı)
- Toplam yıldız 1 + 3 + 5 + 7 + 9 = **n²**: Toplam 25 yıldız (n² = 25)
- Sınır testi n = 1: tek yıldız, sıfır boşluk (Toplam 1 yıldız (n² = 1))

Elmas = piramit + geriye sayan ters piramit

H07_K03_elmas.c · satır 31–41

```
31      for (int i = n - 1; i >= 1; i--) {  /* alt yarı: daralan */
32          for (int b = 1; b <= n - i; b++) {
33              printf(" ");
34          }
35          for (int y = 1; y <= 2 * i - 1; y++) {
36              printf("*");
37              toplam++;
38          }
39          printf("\n");
40      }
41      printf("Toplam %d yıldız\n", toplam);
```

GERÇEK ÇIKTI · N = 4

```
$ ./elmas
> 4
   *
  ***
 *****
*******
 *****
  ***
   *
Toplam 25 yıldız
```

- Üst yarı (satır 21–30): Örnek 4'ün piramidi
- Alt yarı (kodda): aynı formül, dış döngü **geriye** sayar
- n = 4: 16 + 9 = **25** = $n^2 + (n - 1)^2$

02

Problem çözme desenleri

asallık: naif → √n → tek bölenler · seriler ve ε · iç içe döngüden çıkış · menü döngüsü

ÖRNEK 6

Naif asallık: bölen ara, bulunca bayrağı indir ve çık

H07_06_asal_naif.c · satır 20–34

```
20      int asal = 1;   /* bayrak: aksi kanıtlanana dek asal */
21      int bolme = 0;  /* kaç bölme denendi */
22      int bolen = 0;  /* bulunan ilk bölen */
23      if (n < 2) {
24          asal = 0;  /* 0, 1 ve negatifler asal değildir */
25      } else {
26          for (int d = 2; d < n; d++) {
27              bolme++;
28              if (n % d == 0) {
29                  asal = 0;
30                  bolen = d;
31                  break;  /* bir bölen yeter: aramayı bitir */
32              }
33          }
34      }
```

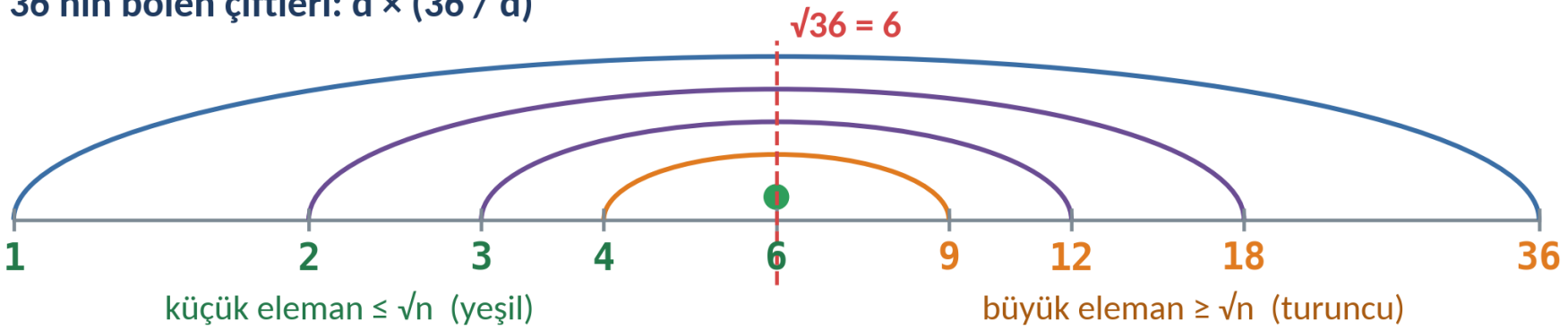
GERÇEK ÇIKTI

```
> 97
97 asaldır (95 bölme denendi).
> 91
91 asal değildir: 7 x 13 (6 bölme denendi).
> 2
2 asaldır (0 bölme denendi).
```

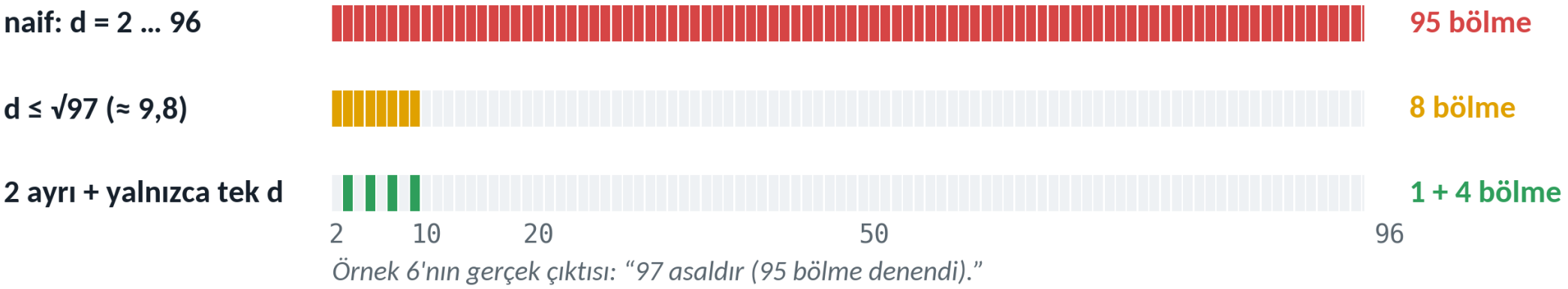
- Bayrak `asal = 1`: aksi kanıtlanana dek asal
- 0, 1, negatif: döngüye girmeden **değil**
- 2: koşul `2 < 2` yanlış → döngü hiç dönmez → asal
- INT_MAX: **2 147 483 645** bölme

Neden √n yeter? Bölen çiftleri aynalanır

a) 36'nın bölen çiftleri: d × (36 / d)



b) 97 asal mı? Her yöntemin denediği bölenler



Örnek 6'nın gerçek çıktısı: "97 asaldır (95 bölme denendi)."

Ders notu Şekil 7.3 · K11 (36'nın çiftleri) ve Örnek 6'nın gerçek çıktısı (97 için 95 bölme)

İyileştirilmiş test: sınırlar önce, sonra tek d ≤ n / d

H07_11_asal_karekok_sure.c · satır 51–65

```
51          if (n < 2) {
52              asal = 0;  /* 0 ve 1 asal değil */
53          } else if (n == 2) {
54              asal = 1;  /* tek çift asal */
55          } else if (n % 2 == 0) {
56              asal = 0;  /* 2'den büyük çiftler */
57          } else {
58              for (int d = 3; d <= n / d; d += 2) {  /* d <= √n, d tek */
59                  iyi_bolme++;
60                  if (n % d == 0) {
61                      asal = 0;
62                      break;
63                  }
64              }
65          }
```

d	n / d	d <= n / d	n % d
3	32	doğru	1
5	19	doğru	2
7	13	doğru	6
9	10	doğru	7
11	**8**	**yanlış**	**–**

ÖRNEK 12

Kaç tur döneceği bilinmeyen döngü: terim < ε olunca dur

H07_12_e_pi_seri.c · satır 35-44

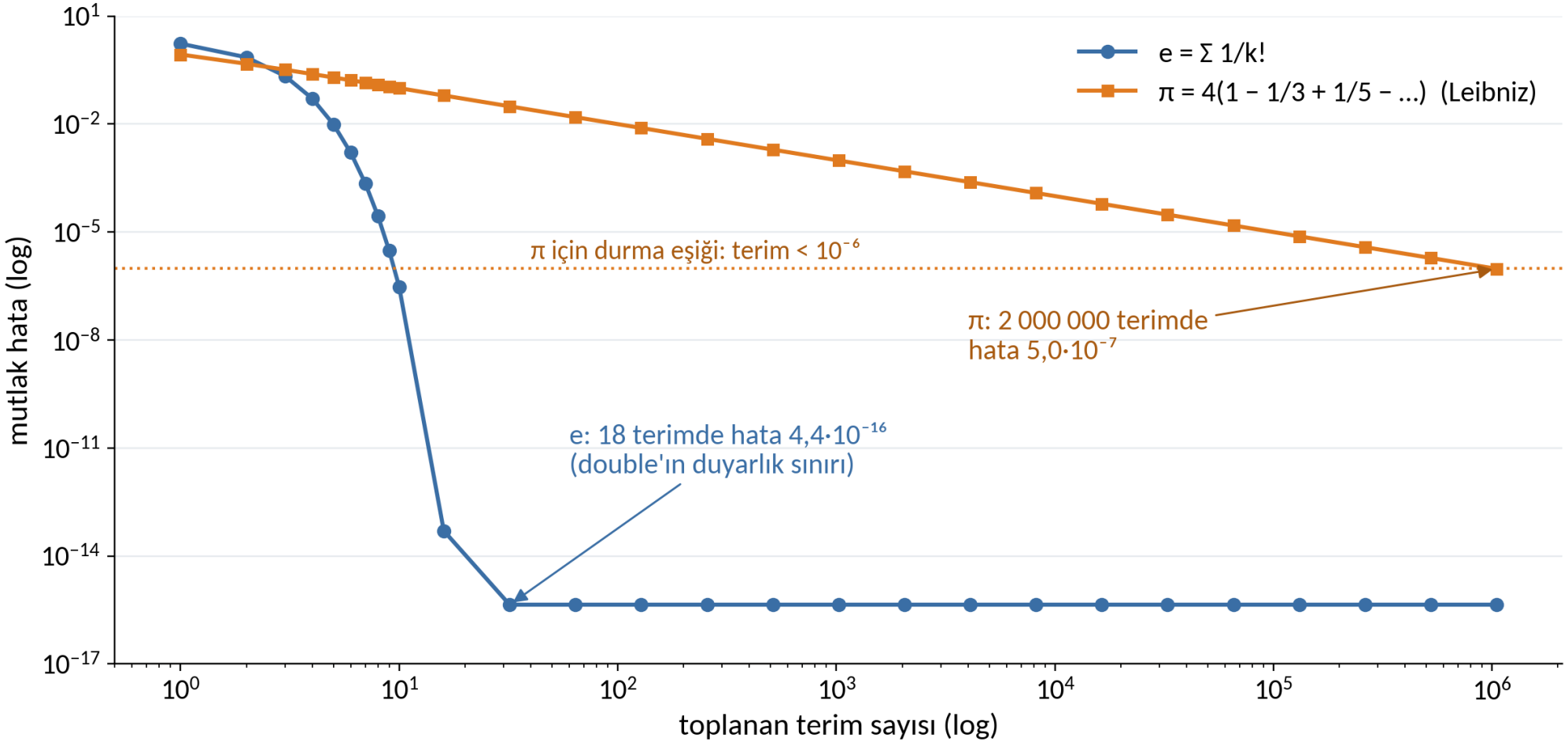
```
35      double pi = 0.0;
36      double isaret = 1.0;
37      double buyukluk = 4.0;  /* 4/(2t+1), t = 0 */
38      int t = 0;
39      while (buyukluk >= EPS_PI) {
40          pi += isaret * buyukluk;
41          isaret = -isaret;  /* + - + - ... */
42          t++;
43          buyukluk = 4.0 / (2 * t + 1);
44      }
```

ÇIKTI

```
e ≈ 2.718281828459046
  18 terim, hata 4.4e-16
π ≈ 3.141592153589724
  2000000 terim, hata 5.0e-07
```

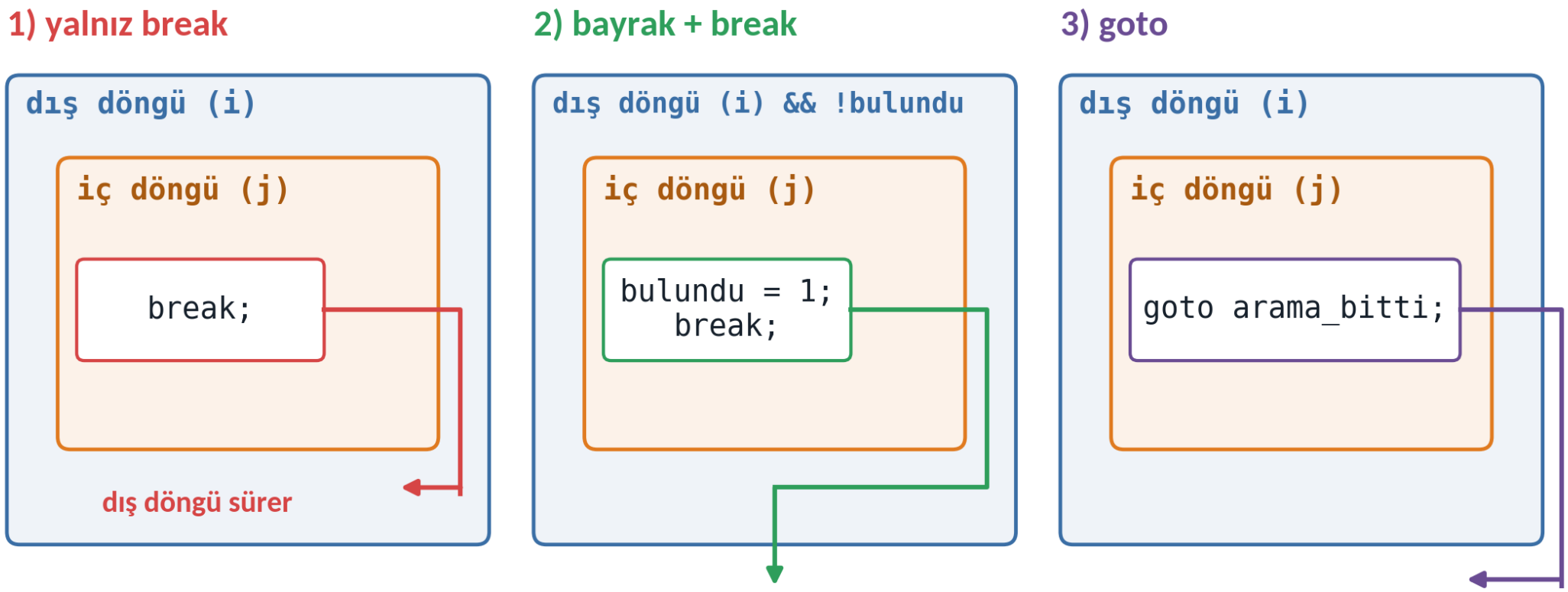
- e: Σ 1/k! — `terim /= k`
- π: Leibniz, işaret her terimde değişir
- `4.0 / …` kesirli bölme (`1 / k` = 0!)
- e **18** terim · π **2 000 000** terim

Yakınsama: e hızla, π çok yavaş



Terim sayısına karşı mutlak hata (log–log); e'nin hatası double'ın duyarlık sınırında (~4,4·10⁻¹⁶) durur

İç içe döngüden çıkış: break yalnızca en içtekinden çıkar



200 deneme · 3 kez “bulundu”

18 deneme · ilk çözümde durur

18 deneme · ilk çözümde durur

$i^2 + j^2 = 325$ için ilk çift ($1^2 + 18^2$): yalnız break 200 deneme ve 3 “bulundu”, bayrak ya da goto 18 deneme

MÜHENDİS GÖZÜYLE

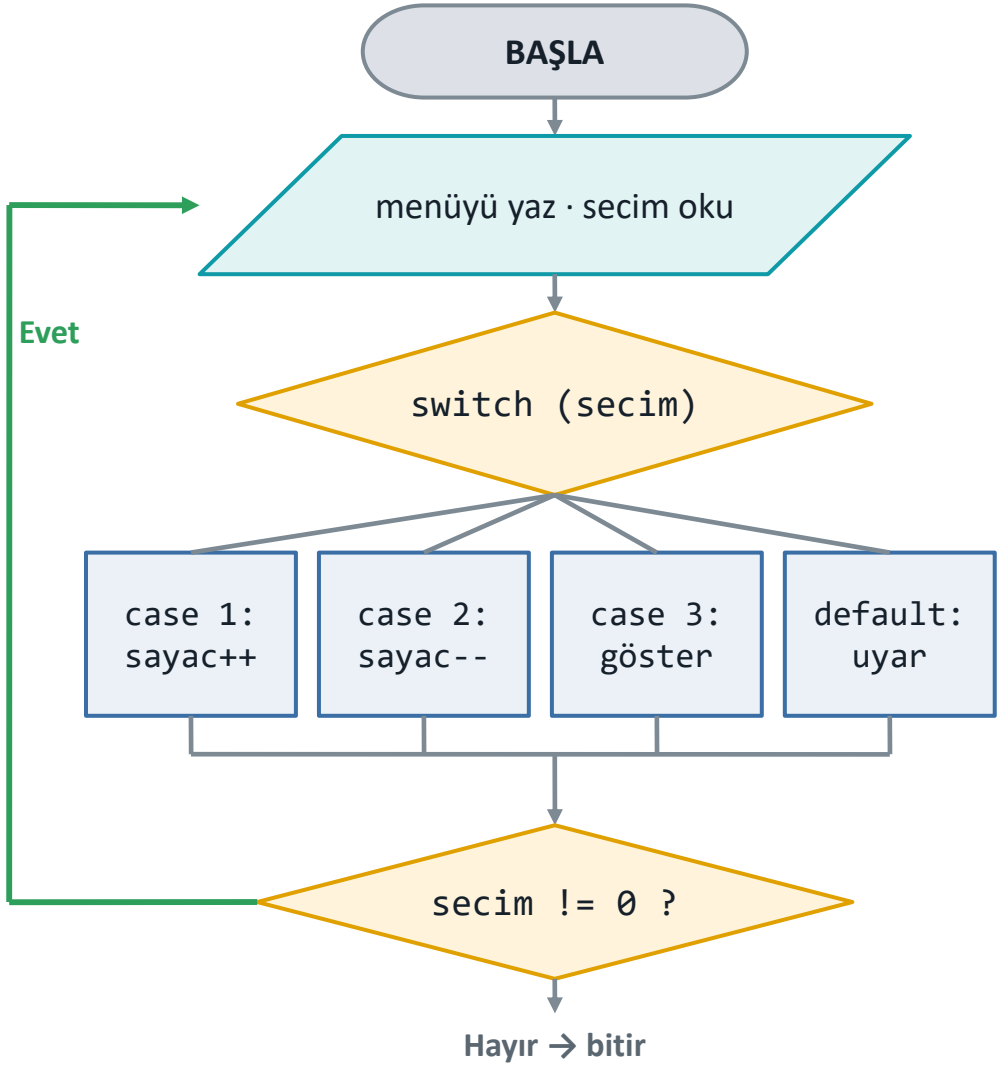
goto: “zararlı” mı, yerinde mi?

- **Dijkstra (1968):** denetimsiz `goto` programı izlemeyi zorlaştırır; yapısal programlamanın yerleşmesinde etkili oldu
- **Knuth (1974):** bazı durumlarda `goto`, yapısal alternatifinden daha okunur olabilir
- C’de kabul gören dar kullanımlar: **iç içe döngüden tek adımda çıkış** ve fonksiyon sonunda ortak **temizlik** (Linux çekirdeği kodlama kılavuzu)
- Bu derste: önce **bayrak** ya da (H8) **fonksiyon + return**; `goto` yalnızca ileri, tek etikete

H07_K04_ic_ice_cikis.c · satır 54–65

```
54      for (int i = 1; i <= UST; i++) {
55          for (int j = i; j <= UST; j++) {
56              deneme++;
57              if (i * i + j * j == HEDEF) {
58                  bi = i;
59                  bj = j;
60                  goto arama_bitti;  /* iki döngüden çık */
61              }
62          }
63      }
64  arama_bitti:
65      if (bi > 0) {
```

Menü döngüsü: case'teki break yalnızca switch'ten çıkar



H07_K12_menu_do_while.c · satır 22–36

```
22          case 1:
23              sayac++;
24              break;  /* yalnızca switch'ten çıkar */
25          case 2:
26              sayac--;
27              break;
28          case 3:
29              printf("sayac = %d\n", sayac);
30              break;
31          case 0:
32              printf("Güle güle.\n");
33              break;
34          default:
35              printf("Geçersiz seçim: %d\n", secim);
36              break;
```

03

Kaputun altında: “daha hızlı”yı ölçmek

clock() ve CLOCKS_PER_SEC · tekrar ve medyan · -O0 / -O2 · ölçülen kod silinebilir

clock(): işlemci zamanını tık olarak ölçmek

H07_K05_clock_temel.c · satır 15–26

```
15      printf("CLOCKS_PER_SEC = %ld\n", (long)CLOCKS_PER_SEC);
16
17      clock_t bas = clock();  /* başlangıç tıkı */
18      long long toplam = 0;
19      for (int i = 1; i <= 100000000; i++) {
20          toplam += i % 7;
21      }
22      clock_t son = clock();  /* bitiş tıkı */
23
24      double sure = (double)(son - bas) / CLOCKS_PER_SEC;
25      printf("toplam = %lld\n", toplam);
26      printf("geçen işlemci zamanı = %.3f s (%ld tık)\n",
```

ÇIKTI

```
$ time ./clock_temel
CLOCKS_PER_SEC = 1000000
toplam = 299999997
geçen işlemci zamanı = 0.278 s (277726 tık)

real 0m0.287s
user 0m0.280s
sys 0m0.000s
```

- `clock()` → işlemci zamanı (tık)
- fark ÷ `CLOCKS_PER_SEC` → saniye
- Sonuç **yazdırılır**: ölçülen kod silinmesin
- Windows (MSVC CRT): duvar saati, 1000 tık/s

KAPUTUN ALTINDA

Ölçüm ilkeleri

Tekrar et, medyan al

Tek ölçüm gürültülüdür. Aynı işi **5 kez** ölç, **medyanı** raporla; çok kısa işi R kez tekrarla

Sonucu kullan

Kullanılmayan hesabı derleyici **silebilir**: sonucu yazdır ya da `volatile`

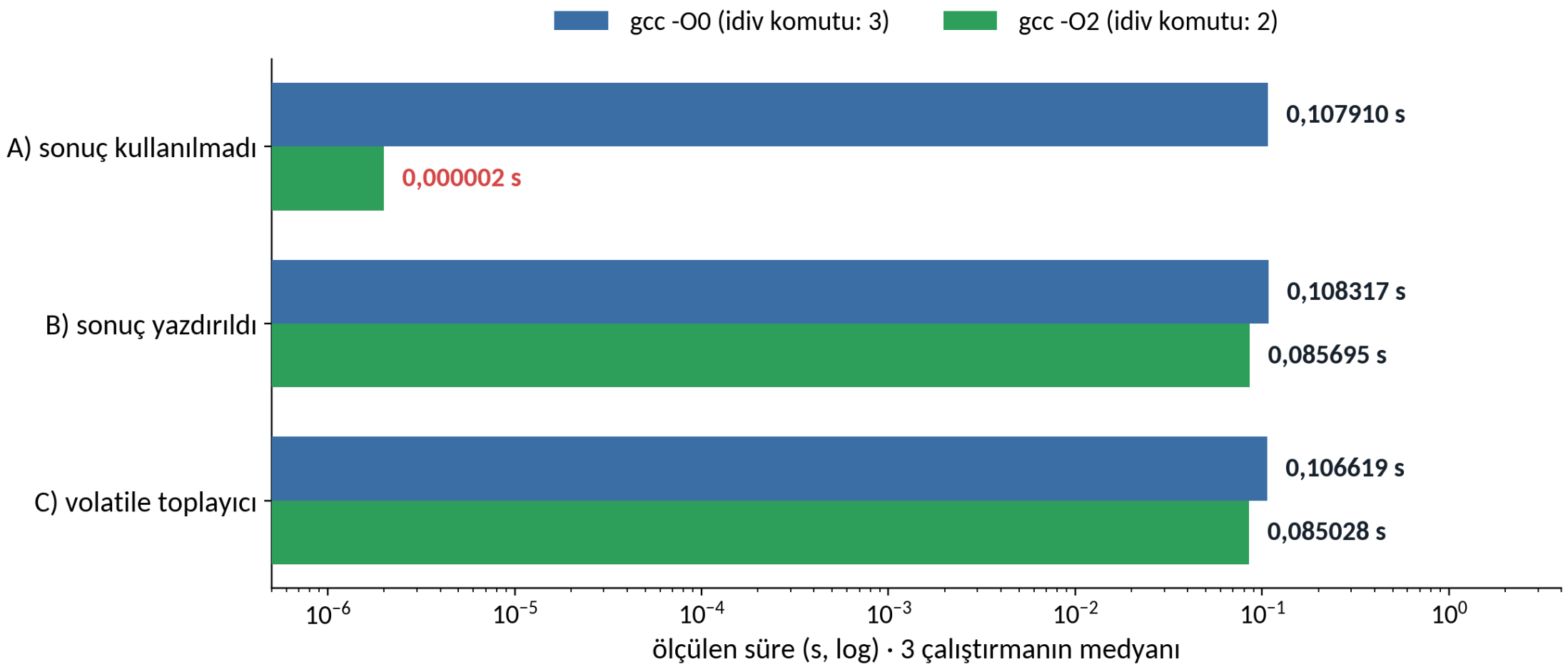
Ortamı yaz

İşlemci, derleyici, **-O0 / -O2**, N, tekrar sayısı, yük

Oranı yorumla

Süre makineyle değişir; **yineleme sayısı değişmez**. Eğilime ve orana bak

Ölçülen kod silinebilir: -O2 ve kullanılmayan sonuç



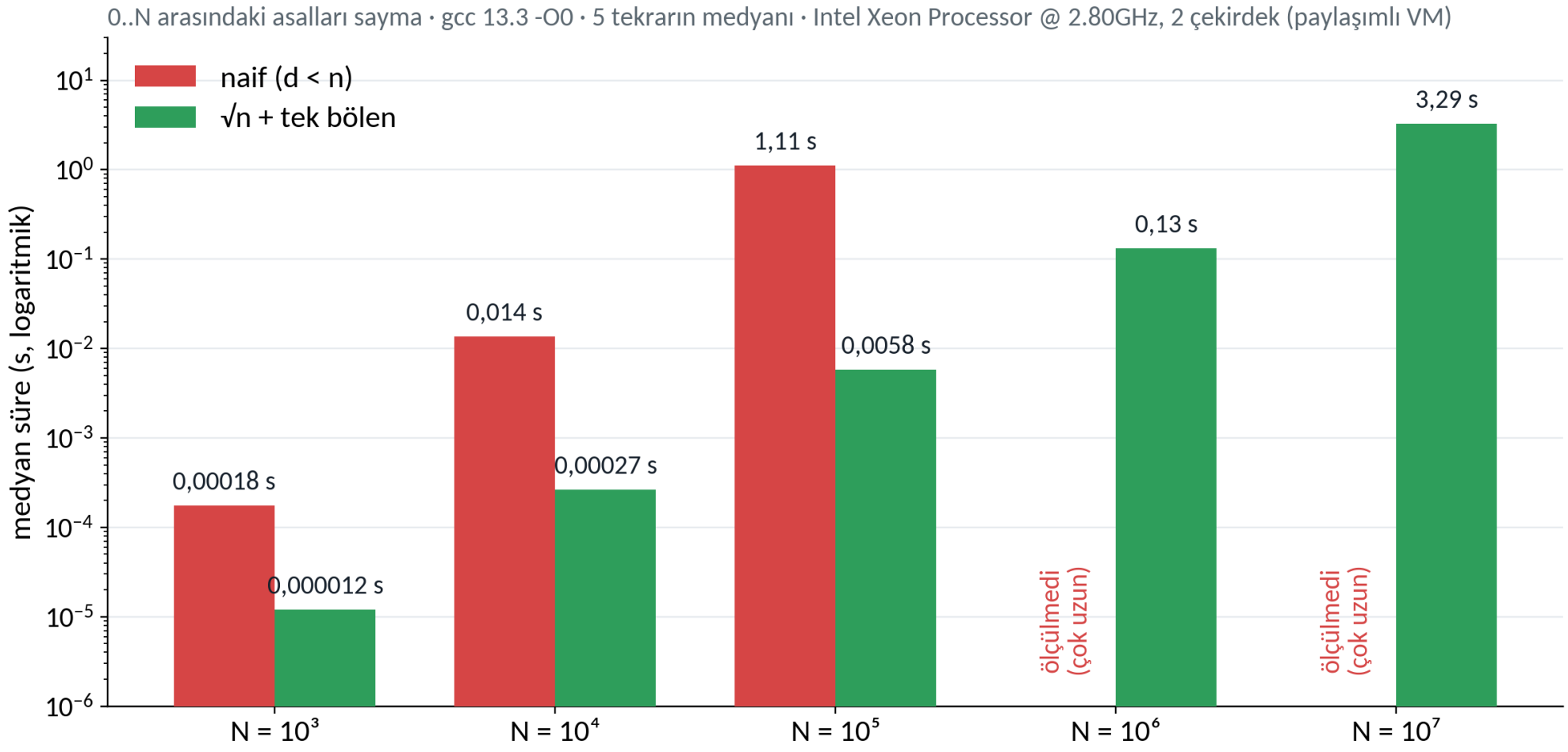
objdump: -O0 sürümünde 3, -O2 sürümünde 2 idiv komutu — A döngüsü makine kodundan tümüyle silinmiş

Ölçüm deneyi: 0..N arasındaki asalları say (5 tekrarın medyanı)

gcc 13.3 -O0 · Intel Xeon Processor @ 2.80GHz, 2 çekirdek, **paylaşımlı sanal makine** · süreler göreli okunmalı

N	asal	naif: bölme	naif: süre (s)	iyileşt.: bölme	iyileşt.: süre (s)
10^3	168	78 022	0,000176	2 351	0,000012
10^4	1 229	5 775 223	0,0137	55 958	0,000266
10^5	**9 592**	**455 189 150**	**1,1114**	**1 345 442**	**0,005778**
10^6	78 498	–	ölçülmedi	33 600 560	0,1312
10^7	664 579	–	ölçülmedi	870 439 156	3,2945

Süreler: naif ve iyileştirilmiş asallık



Ders notu Şekil 7.5 · logaritmik eksen: her çizgi on kat

Haftanın 14 basamağı

#	Dosya	Yeni öge	Düzey
1	`H07_01_dikdortgen.c`	İç içe döngünün temel yapısı	★
2	`H07_02_ucgenler.c`	İç döngü sınırının dış değişkene bağlanması	★
3	`H07_03_carpim_tablosu.c`	Hizalı tablo çıktısı	★
4	`H07_04_piramit.c`	Satır başına formül çıkarma	★★
5	`H07_05_floyd_ucgeni.c`	Desen içinde sayaç sürdürme	★★
6	`H07_06_asal_naif.c`	Bölen arama; bayrak	★★
7	`H07_07_asallar_1_1000.c`	İç içe döngü ile listeleme	★★
8	`H07_08_mukemmel_armstrong.c`	Bölen toplamı; basamak üsleri	★★
9	`H07_09_fibonacci.c`	Kayan değişkenler (önceki iki terim)	★★
10	`H07_10_onluk_ikili.c`	Ardışık bölme; tersten yazdırma sorunu (H9/H10'a köprü)	★★
11	`H07_11_asal_karekok_sure.c`	Verimlilik iyileştirmesi; clock()	★★★
12	`H07_12_e_pi_seri.c`	ε eşiğiyle sonlanan seri; yakınsama	★★★
13	`H07_13_pisagor_ucluleri.c`	Arama uzayı; iç içe üç döngü	★★★
14	`H07_14_atm_menu.c`	Bütünleştirme: döngü + switch + doğrulama	★★★

ÖRNEK 3 ★

Örnek 3: tek döngüyü bir dış döngüye koymak → 10 × 10 tablo

H07_03_carpim_tablosu.c · satır 24–30

```c
    for (int i = 1; i <= N; i++) {  /* dış döngü: satır (i) */
        printf("%4d |", i);
        for (int j = 1; j <= N; j++) {  /* iç döngü: sütun (j) */
            printf("%4d", i * j);
        }
        printf("\n");
    }
```

GERÇEK ÇIKTI

```
$ ./carpim_tablosu
   x |   1   2   3   4   5   6   7   8   9  10
-----+----------------------------------------
   1 |   1   2   3   4   5   6   7   8   9  10
   2 |   2   4   6   8  10  12  14  16  18  20
   3 |   3   6   9  12  15  18  21  24  27  30
   4 |   4   8  12  16  20  24  28  32  36  40
   5 |   5  10  15  20  25  30  35  40  45  50
   6 |   6  12  18  24  30  36  42  48  54  60
   7 |   7  14  21  28  35  42  49  56  63  70
   8 |   8  16  24  32  40  48  56  64  72  80
   9 |   9  18  27  36  45  54  63  72  81  90
  10 |  10  20  30  40  50  60  70  80  90 100
```

- H6'daki tek sayının tablosu = **iç** döngü; onu i ile tekrarlayan **dış** döngü
- `%4d`: her hücre 4 karakter → sütunlar hizalı
- `printf("\n")` iç döngünün **dışında**, dış döngünün **içinde**
- 100 hücre = 10 × 10 yineleme

ÖRNEK 5 ★★

Floyd üçgeni: satırlar boyunca süren sayaç

H07_05_floyd_ucgeni.c · satır 21–29

```
21      int sayi = 1;  /* satırlar boyunca süren sayaç */
22      for (int i = 1; i <= n; i++) {
23          for (int j = 1; j <= i; j++) {
24              printf("%4d", sayi);
25              sayi++;
26          }
27          printf("\n");
28      }
29      printf("Son sayı: %d = n(n+1)/2\n", sayi - 1);
```

ÇIKTI

```
Floyd üçgeni:
   1
   2   3
   4   5   6
   7   8   9  10
  11  12  13  14  15
Son sayı: 15 = n(n+1)/2
```

- `sayi` dış döngüden **önce** başlar, hiç sıfırlanmaz
- Son sayı = yineleme sayısı = n(n+1)/2
- Sayı üçgeninde yazılan `j`: her satırda yeniden 1

ÖRNEK 8 ★★

Mükemmel ve Armstrong sayıları: üç katlı iç içe döngü

H07_08_mukemmel_armstrong.c · satır 41–54

```
41          int k = 0;  /* basamak sayısı */
42          for (int s = n; s > 0; s /= 10) {
43              k++;
44          }
45          int toplam = 0;
46          for (int s = n; s > 0; s /= 10) {  /* her basamak için */
47              int basamak = s % 10;
48              int kuvvet = 1;
49              for (int e = 1; e <= k; e++) {  /* basamak^k: tekrarlı çarpma */
50                  kuvvet *= basamak;
51              }
52              toplam += kuvvet;
53          }
54          if (toplam == n) {
```

GERÇEK ÇIKTI (KESİT)

```
6 = 1 + 2 + 3
28 = 1 + 2 + 4 + 7 + 14
496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248
8128 = 1 + 2 + … + 4064
1-99999 arasındaki Armstrong sayıları:
1 2 3 4 5 6 7 8 9 153 370 371 407 1634 8208 9474 54748 92727 93084
```

- Aday → basamak → kuvvet: **üç** katlı döngü
- Toplayıcı her aday için **0**'dan
- pow yerine tekrarlı çarpma (tamsayı)

Fibonacci: kayan pencere ve taşma sınırı

Yalnızca son iki terim tutulur: her turda pencere bir adım sağa kayar

F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	F(6)	F(7)	F(8)	F(9)
0	1	1	2	3	5	8	13	21	34

`onceki simdiki` tur t

`onceki simdiki` tur t+1

Toplamadan ÖNCE denetim: `simdiki <= INT_MAX - onceki`

```
int sonraki = onceki + simdiki;
onceki = simdiki;
simdiki = sonraki;
```

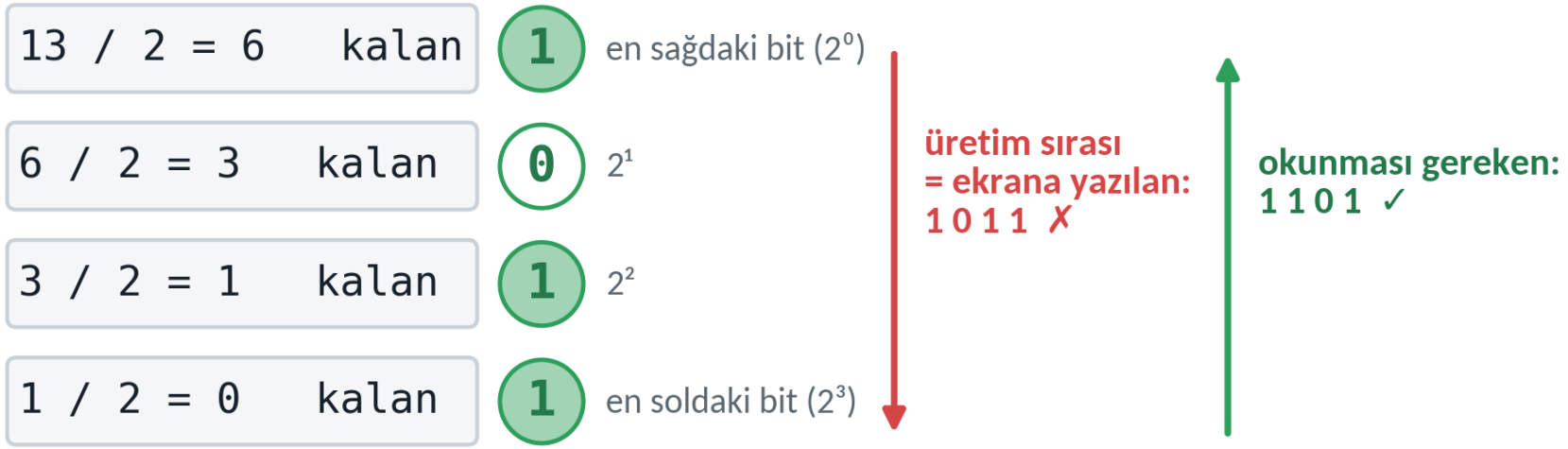
int (32 bit): F(46) = 1 836 311 903 ✓ F(47) > INT_MAX ✗

unsigned long long (64 bit): F(93) ✓ F(94) ✗ (2^{64} modunda sarar)

-fsanitize=undefined ile int sürümü: runtime error: signed integer overflow: 1134903170 + 1836311903 cannot be represented in type 'int'

Onluk → ikili: tersten yazdırma sorunu

a) Ardışık 2'ye bölme (13)



Tersten yazdırma sorunu: kalanları saklayıp ters sırada yazmak gerekir → H9 (özyineleme) · H10 (dizi)

b) Dizisiz geçici çözüm: n'yi aşmayan en büyük 2 kuvvetinden aşağı



13 için program "1011" üretir (doğrusu 1101); geçici çözümle 2026 → 11111101010

ÖRNEK 11 ★★★ · N = 100 000

Örnek 11: iki yöntem, aynı sonuç, farklı iş

H07_11_asal_karekok_sure.c · satır 70–75

```
70      printf("  asal          bölme   süre (s)  yöntem\n");
71      printf("%6d %14lld %10.4f  naif (d < n)\n",
72             naif_adet, naif_bolme, naif_sn);
73      printf("%6d %14lld %10.4f  √n + yalnızca tek bölenler\n",
74             iyi_adet, iyi_bolme, iyi_sn);
75      printf("Bölme oranı: %.0f kat daha az yineleme\n",
```

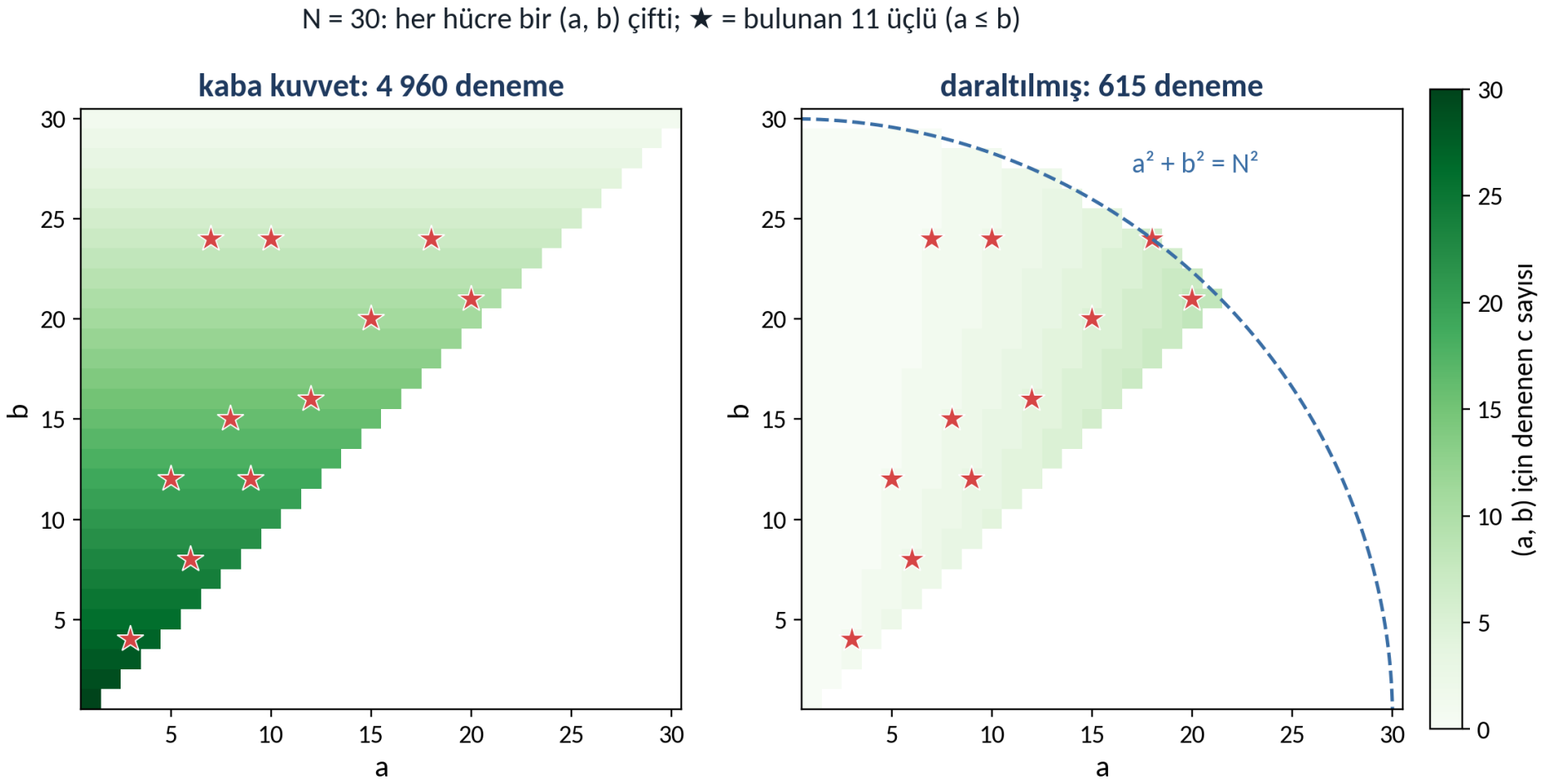
GERÇEK ÇIKTI · N = 100 000

```
  asal          bölme   süre (s)  yöntem
  9592      455189150     1.1120  naif (d < n)
  9592        1345442     0.0060  √n + yalnızca tek bölenler
Bölme oranı: 338 kat daha az yineleme
```

- İki yöntem **aynı** asal sayısını vermeli: doğruluk önce
- Bölme oranı **338** (her makinede aynı)
- Süreler her çalıştırmada biraz değişir
- Yazdırma ölçümün **dışında**

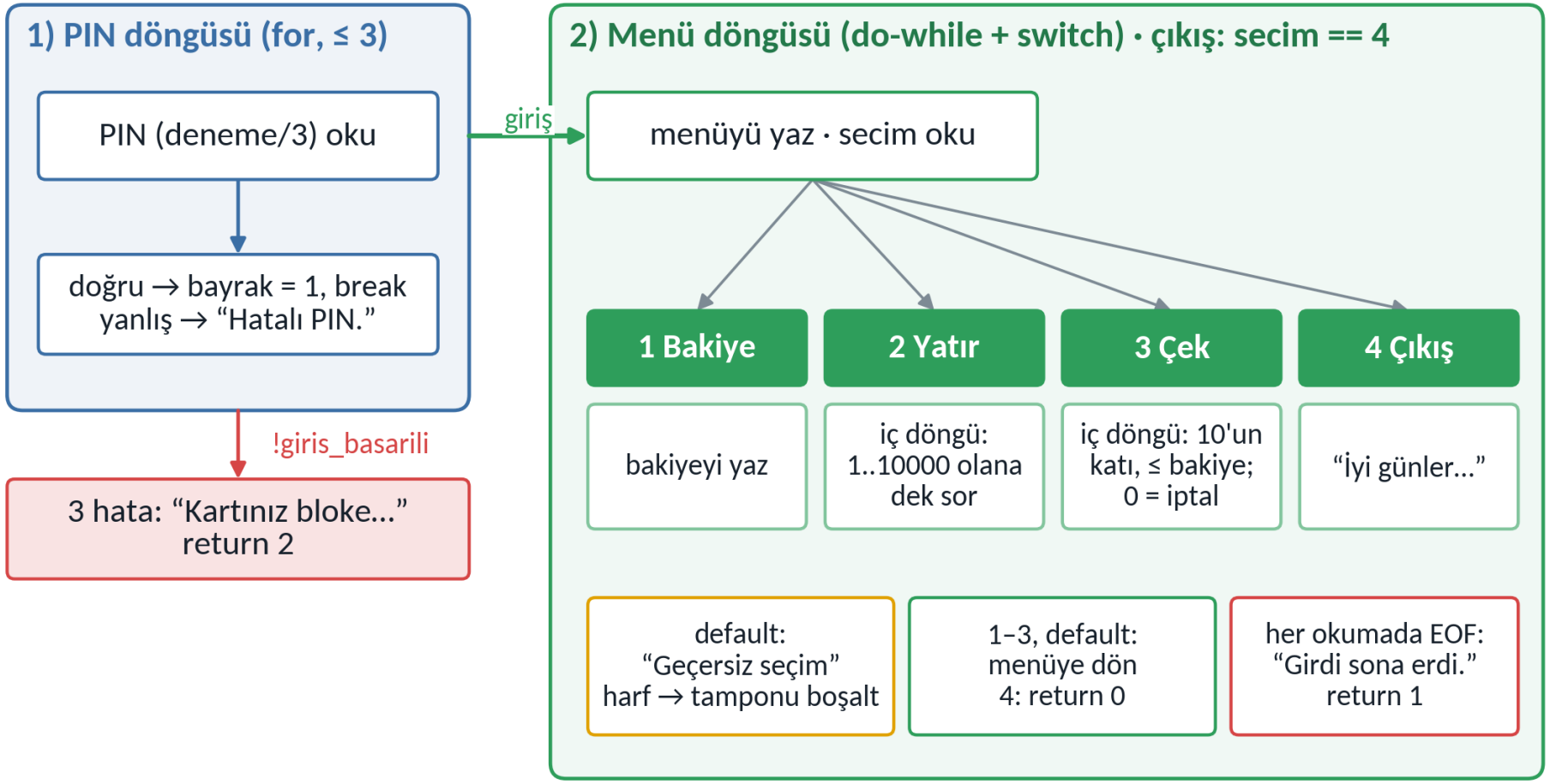
Değiştir: yalnızca **6k ± 1** biçimindeki bölenleri deneyin; bölme sayısı ne kadar azalır?

Pisagor üçlüleri: kaba kuvvet ve daraltılmış arama uzayı



N = 500: kaba kuvvet 20 958 500 deneme, daraltılmış 3 228 813 deneme; ikisi de 386 üçlü

Örnek 14 · ATM: iki döngü, içinde doğrulama döngüleri



PIN döngüsü (for + bayrak + break) → menü döngüsü (do-while + switch); case'lerin içinde doğrulama döngüleri

ATM: PIN döngüsü — sayaç kontrollü for + bayrak

H07_14_atm_menu.c · satır 36–50

```
36          if (okunan == 0) {
37              while ((c = getchar()) != '\n' && c != EOF) {
38                  /* satırın kalanını at */
39              }
40              pin = -1;
41          }
42          if (pin == DOGRU_PIN) {
43              giris_basarili = 1;
44              break;                          /* for döngüsünden çık */
45          }
46          printf("  Hatalı PIN.\n");
47      }
48      if (!giris_basarili) {
49          printf("Kartınız bloke edildi. Lütfen şubenize başvurun.\n");
50          return 2;
```

GERÇEK ÇIKTI

```
$ ./atm
===== AP-1 BANKASI ATM =====
PIN (1/3)           : 1111
  Hatalı PIN.
PIN (2/3)           : 2222
  Hatalı PIN.
PIN (3/3)           : abc
  Hatalı PIN.
Kartınız bloke edildi. Lütfen şubenize
başvurun.
[çıkış kodu: 2]
```

- Kesit, satır 29 ile başlayan `for` döngüsünün (en çok 3 deneme) sonudur
- Harf (`abc`) → `pin = -1` → yanlış deneme
- Bu `break` **for**'dan çıkar; menüdeki yalnızca **switch**'ten

04

Hata Avcısı

sıfırlanmayan iç toplayıcı · aynı sayaç · break’in kapsamı · 0, 1, 2 · 1 / k · d * d taşması

HA1 · İç toplayıcı her dış turda sıfırlanmalı

hata_avcisi/H07_HA1_ic_toplayici.c · satır 12–18

```c
12      int satir_toplami = 0;  /* yalnızca BİR KEZ sıfırlanıyor! */
13      for (int i = 1; i <= 4; i++) {
14          for (int j = 1; j <= 5; j++) {
15              satir_toplami += i * j;
16          }
17          printf("%d. satırın toplamı: %d\n", i, satir_toplami);
18      }
```

Hatalı: toplayıcı döngüden önce bir kez

GERÇEK ÇIKTI

```
1. satırın toplamı: 15
2. satırın toplamı: 45
3. satırın toplamı: 90
4. satırın toplamı: 150
```

hata_avcisi/H07_HA1_ic_toplayici_duzeltilmis.c · satır 12–18

```c
12      for (int i = 1; i <= 4; i++) {
13          int satir_toplami = 0;  /* her dış turda yeniden 0 */
14          for (int j = 1; j <= 5; j++) {
15              satir_toplami += i * j;
16          }
17          printf("%d. satırın toplamı: %d\n", i, satir_toplami);
18      }
```

Düzeltilmiş: dış döngünün içinde

GERÇEK ÇIKTI

```
1. satırın toplamı: 15
2. satırın toplamı: 30
3. satırın toplamı: 45
4. satırın toplamı: 60
```

İlk satır iki sürümde de **15**: hatayı görmek için **ikinci satırı** test edin

HA2 · aynı sayaç · HA3 · break'in kapsamı

hata_avcisi/H07_HA2_ayni_sayac.c · satır 12–19

```
12      int i;
13      for (i = 1; i <= 3; i++) {
14          for (i = 1; i <= 5; i++) {  /* HATA: aynı sayaç! */
15              printf("*");
16          }
17          printf("\n");
18      }
19      printf("Döngüler bitti: i = %d\n", i);
```

HA2: iç döngü de i'yi kullanıyor

GERÇEK ÇIKTI

```
*****
Döngüler bitti: i = 7
```

hata_avcisi/H07_HA3_break_ic.c · satır 15–24

```
15      for (int i = 1; i <= 20; i++) {
16          for (int j = i; j <= 20; j++) {
17              if (i * i + j * j == HEDEF) {
18                  bi = i;
19                  bj = j;
20                  break;  /* "aramayı bitirir" sanılıyor */
21              }
22          }
23      }
24      printf("İlk çözüm: %d^2 + %d^2 = %d\n", bi, bj, HEDEF);
```

HA3: break yalnızca içten çıkar

GERÇEK ÇIKTI

```
İlk çözüm: 10^2 + 15^2 = 325
```

HA2 sınırlar ters olsaydı **sonsuz döngü**: 1 s'de 39 927 808 satır (timeout) · HA3 düzeltme: `i <= 20 && !bulundu` → $1^2 + 18^2$

HA4 · 0, 1, 2 sınırları · HA5 · 1 / k tamsayı bölmesi

hata_avcisi/H07_HA4_asal_sinirlar.c · satır 13–24

```
13      for (int n = -3; n <= 30; n++) {
14          int asal = 1;
15          if (n % 2 == 0) {
16              asal = 0;  /* çiftler asal değil... 2 de mi? */
17          } else {
18              for (int d = 3; d <= n / d; d += 2) {
19                  if (n % d == 0) {
20                      asal = 0;
21                      break;
22                  }
23              }
24          }
```

HA4: n < 2 yok; çiftler 2'den önce

HATALI · GERÇEK ÇIKTI

```
…: -3 -1 1 3 5 7 11 13 17 19 23 29
```

DÜZELTİLMİŞ: N < 2, N == 2, SONRA ÇİFTLER

```
…: 2 3 5 7 11 13 17 19 23 29
```

hata_avcisi/H07_HA5_tam_bolme.c · satır 13–18

```
13      for (int k = 0; k <= 15; k++) {
14          if (k > 0) {
15              faktoriyel *= k;  /* k! */
16          }
17          e += 1 / faktoriyel;  /* HATA: int / long long */
18      }
```

HA5: int / long long → 0

HATALI VE 1.0 / FAKTORİYEL İLE

```
$ ./ha5
e ≈ 2.000000000000000
$ ./ha5d
e ≈ 2.718281828458995
```

HA6 · d * d <= n taşar: 2 147 483 647 “asal değil”!

hata_avcisi/H07_HA6_karekok_tasma.c · satır 17–24

```
17      for (int d = 3; d * d <= n; d += 2) {  /* HATA: d * d taşabilir */
18          bolme++;
19          if (n % d == 0) {
20              asal = 0;
21              bolen = d;
22              break;
23          }
24      }
```

Hatalı: d = 46 341’de d * d taşar

GERÇEK ÇIKTI

```
2147483647: asal = 0, bölen =
2147483647, 1073741823 bölme
```

hata_avcisi/H07_HA6_karekok_tasma_duzeltilmis.c · satır 17–24

```
17      for (int d = 3; d <= n / d; d += 2) {  /* taşmasız √n sınırı */
18          bolme++;
19          if (n % d == 0) {
20              asal = 0;
21              bolen = d;
22              break;
23          }
24      }
```

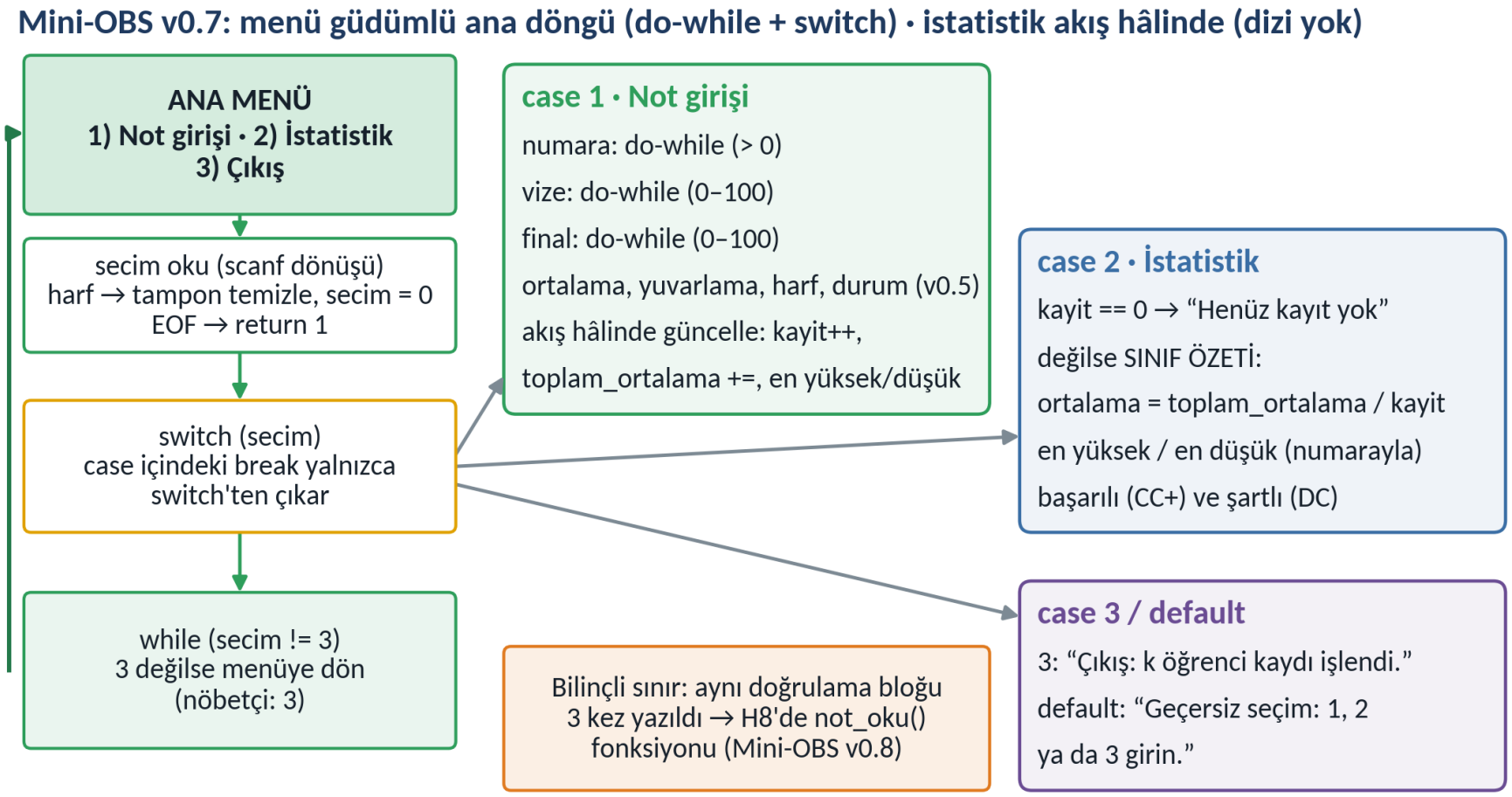
Düzeltilmiş: d <= n / d

GERÇEK ÇIKTI

```
2147483647: asal = 1, bölen = 0, 23169
bölme
```

UBSan: `runtime error: signed integer overflow: 46341 * 46341 cannot be represented in type 'int'` · tanımsız davranış: bu makinede “sardı”, başka yerde başka şey olabilir

Mini-OBS v0.7: menü güdümlü ana döngü



T1 gerçek çıktısı: 4 kayıt, sınıf ortalaması 68.20, başarılı 2 / 4 · 5 test: 91_Proje_MiniOBS/v0.7/testler

MİNİ-OBS PROJESİ

Ara proje (eşli): menü güdümlü bir program

- **A) ATM:** bakiye, yatırma/çekme, **3 hatalı PIN'de kilit**, her girdi doğrulanır
- **B) Mini-OBS v0.7:** menü güdümlü ana döngü, doğrulama, akış hâlinde istatistik
- **Teslim:** tek .c (sıfır uyarı) · ≥ 6 test ve gerçek çıktıları · 1–2 sayfa rapor
- **Rubrik (100):** doğruluk 35 · sağlamlık 20 · kod kalitesi 15 · testler 15 · rapor 10 · ek 5
- **Dürüstlük:** iki kişi de her satırı açıklayabilmeli; kaynaklar ve yapay zekâ kullanımı raporda

Test tablosu örneği: Örnek 14, gerçek çalıştırmalar

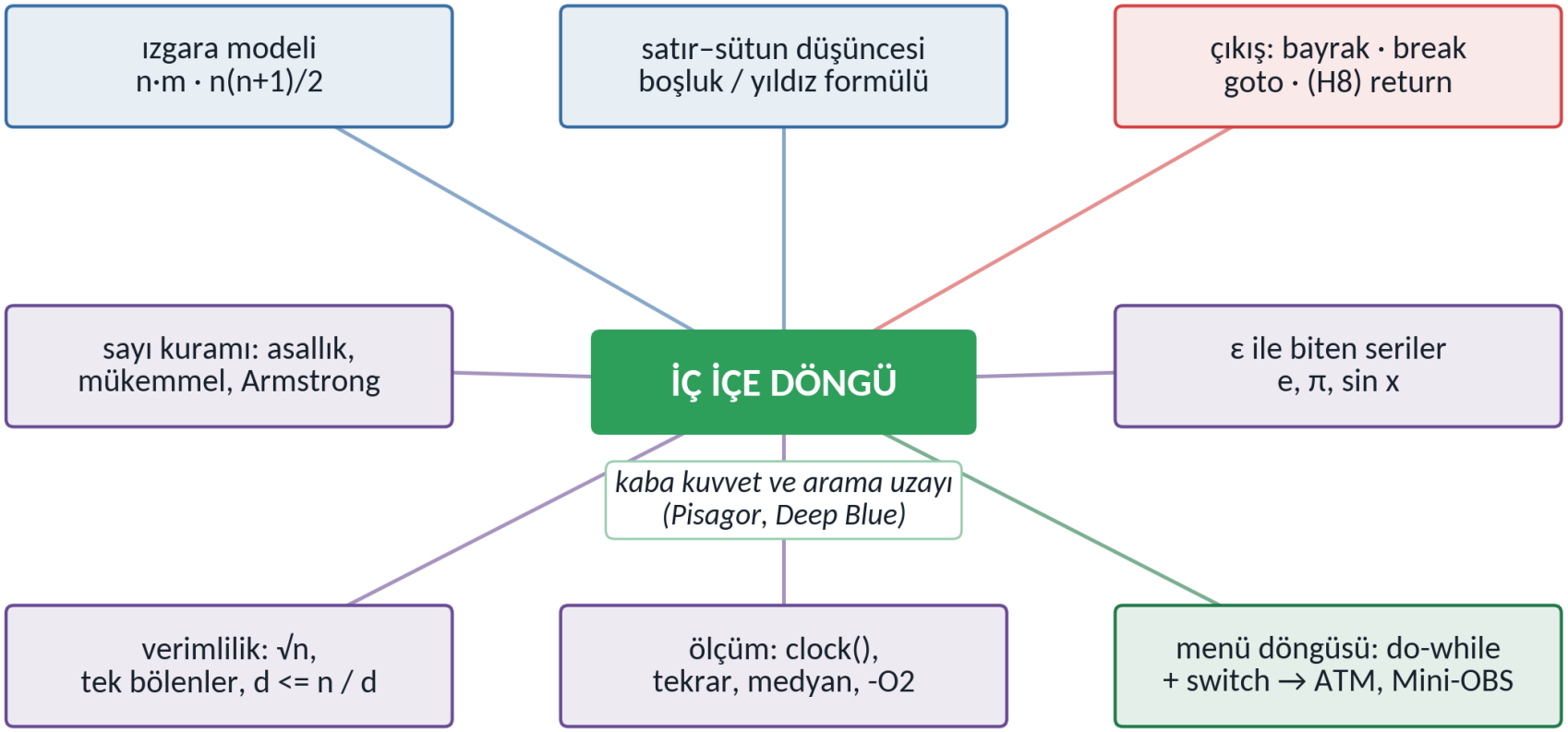
Senaryo	Girdiler	Kod
normal akış	`1111 1234 1 3 250 2 100 4`	`0`
üç hatalı PIN	`1111 2222 abc`	`2`
geçersiz girdiler, yetersiz bakiye, iptal	`1234 7 x 2 0 20000 500 3 15 -10 5000 0 1 4`	`0`
bakiyenin tamamını çekme; sonra çekilebilir tutar yok	`1234 3 1000 3 4`	`0`
girdi yarıda biter (Ctrl+D)	`1234 2`	`1`

AP-2 KÖPRÜSÜ

AP-2 köprüsü: iç içe döngüden algoritma çözümlemesine

Bu hafta (AP-1)	AP-2'de
Yineleme sayımı: N^2 ve $N(N+1)/2$	Kabarcık ve seçme sıralaması: **karesel** karmaşıklık
√n ve tek bölenlerle asallık	**Eratosthenes Kalburu** (diziler sonrası)
Pisagor'da `break` ile arama uzayını daraltma	**Geri izleme** ve **budama**
Sayaç + `clock()` ile ölçüm	Biçimsel çözümleme: **O, Ω, Θ**

Özet ve Büyük Sorunun yanıtı



En hızlı nasıl? Gereksiz bölmeyi yapmayarak: √n'de dur, yalnızca tek bölenler. INT_MAX: naif **2 147 483 645**, iyileştirilmiş **23 169** bölme.

Nasıl ölçeriz? Yineleme sayısı (sayaç) ve süre (`clock()`, tekrar, medyan, ortam). N = 10^5: bölme **338 kat**, süre ~**192 kat** azaldı.

HAFTAYA KÖPRÜ

H8’e köprü: aynı kodu kaç kez yazdık?

12

dosyada bölen arama döngüsü

17

kez aynı denetim: `n % d == 0`

4 + 4

kez `scanf` doğrulama bloğu: ATM ve Mini-OBS

- Bir hatayı (ör. HA4) düzeltmek için **hepsini** bulmak gerekir
- H8: `is_prime(n)` bir kez yazılır, her yerde çağrılır, bir kez test edilir → **fonksiyonlar**

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `for (i = 1; i <= 4; i++) for (j = i; j <= 4; j++) s++;` — `s` kaç olur?

2 **n = 6** yükseklikli piramidin 4. satırında kaç boşluk, kaç yıldız var?

3 Bir menü döngüsünde `case 3:` içindeki `break` neyi bitirir? Döngüyü ne bitirir?

Haftaya hazırlık ve kaynaklar

- **Oku:** ders notu Bölüm 3–5; Deitel ve Deitel (2022) Bölüm 4 (program denetimi)
- **Web:** desen oluşturucu, yineleme sayacı, ölçüm grafiği ve **H1–H7 vize hazırlık soru bankası**
- **Alıştırmalar:** 7.1–7.6 atölyede; 7.7–7.12 ödev · **ara proje** yönergesi: 92_Degerlendirme
- **Mini-OBS:** v0.7'yi T1–T5 ile çalıştır; bir test senaryosu ekle
- **Kaynaklar:**
  - Campbell, Hoane & Hsu (2002). Deep Blue. AI 134 · IBM (t.y.). Deep Blue · Shannon (1950)
  - Dijkstra (1968). Go to statement considered harmful · Knuth (1974) · Linux kernel coding style